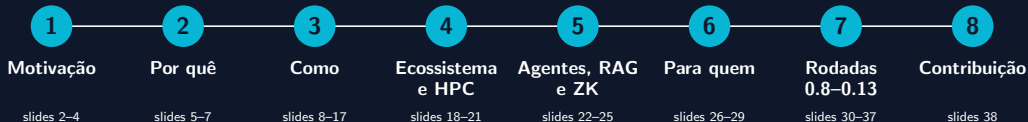
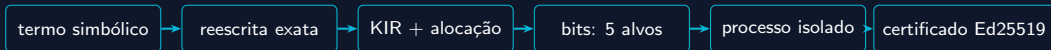


vapor

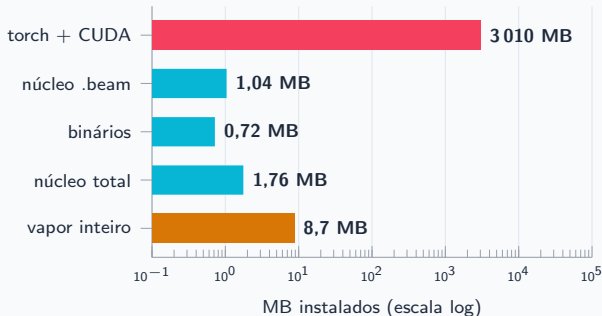
tensores certificados, do termo algébrico ao registrador

Compilador e runtime de tensores em Elixir, deps: []. Emite **bits** para x86-64 (AVX2, AVX-512), AArch64, RVV 1.0 e SPIR-V, executa em processos isolados, sai de cada compilação com um certificado — e sustenta um LLM inteiro com os mesmos bits em todo lugar.



Dor 1 — gigabytes de pilha que ninguém audita

1 · MOTIVAÇÃO



torch + CUDA: torch==2.14.0 e 28 dependências (cp312, manylinux x86_64), resolvidas com `pip install --dry-run` no PyPI em 28/09/2026, wheels comprimidos. vapor (0.13): .beam do núcleo (álgebra, compilação, emissores, verificação, runtime); binários = vapor-worker + vapor-fabric; inteiro = as 13 rodadas (modelos, documentos, mesas, finanças). Nenhum lado conta o runtime da linguagem (Python; Erlang/OTP = 32 MB aqui).

Linhas que alguém precisa ler

Elixir (núcleo)	12 853
Zig (worker + daemon)	4 717
Lean 4 (provas + extrator)	1 515
total do núcleo	19 085

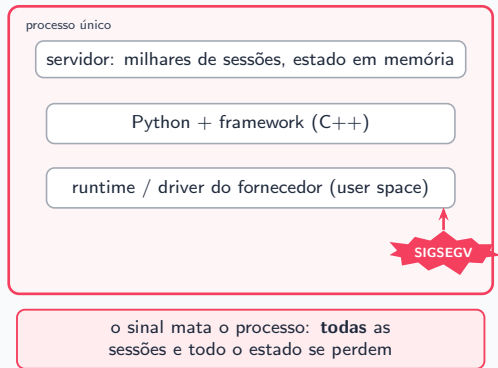
Pilha tradicional: Python, framework C++, compiladores (LLVM, NVCC, Triton), bibliotecas fechadas do fornecedor — **milhões** de linhas na base de confiança.

Cada linha que executa é uma linha em que alguém confia. Escopo menor, sim — mas **~1 700× menos bytes no núcleo** (~350× com as 13 rodadas) e uma base inteira que cabe numa revisão de código.

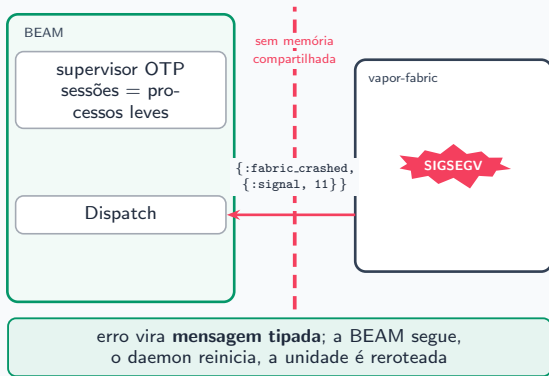
Dor 2 — o driver cai, o serviço inteiro cai junto

1 · MOTIVAÇÃO

modelo usual: um espaço de endereçamento



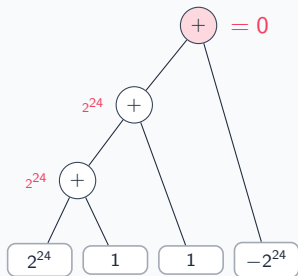
vapor: fronteira de processo



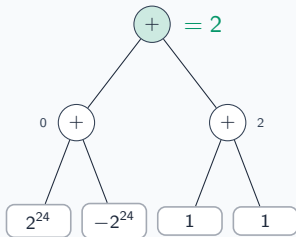
Contenção não é try/catch: é um espaço de endereçamento separado. Fora dele, uma falha de driver é só um dado.

Dor 3 — a mesma soma, bits diferentes

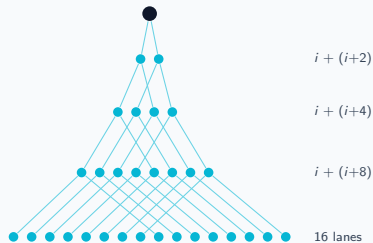
sequencial: $((a + b) + c) + d$



em pares: $(a + d) + (b + c)$



vapor :canonical: a árvore faz parte da especificação



a mesma árvore em x86 (2×8 lanes), NEON (4×4), RVV (VLEN 128–512) e subgrupo Vulkan: **bits idênticos ao oráculo** nos 5 substratos testados

binary32 tem 24 bits de mantissa: $2^{24} + 1$ não é representável e arredonda para 2^{24} . A ordem da redução muda com VLEN, número de threads e driver $\Rightarrow$ o resultado muda com a máquina.

Em ponto flutuante, $(a + b) + c \neq a + (b + c)$. Quem não fixa a ordem entrega um número que depende do hardware; o vapor fixa a ordem e **testa a igualdade de bits** entre máquinas.

Cinco axiomas em vez de mais uma camada

2 · POR QUÊ



fundação: especificação executável — o oráculo binary32 exato na BEAM define o que "correto" significa

Empacotar mais bibliotecas aumenta a superfície confiada sem verificação. Cada axioma **remove uma classe inteira de falha** — e é checado por máquina a cada compilação.



Provado — 82 teoremas, só o core do Lean, zero axiom/sorry (teste de auditoria)

sem wrap em $\mathbb{Z}/2^{32}$ · stride coprimo sem conflito de banco · monoide afim segmentado · soundness do checker de alocação · monotonia do envelope

Testado contra quem não compartilha código conosco — os níveis diferenciais da suíte

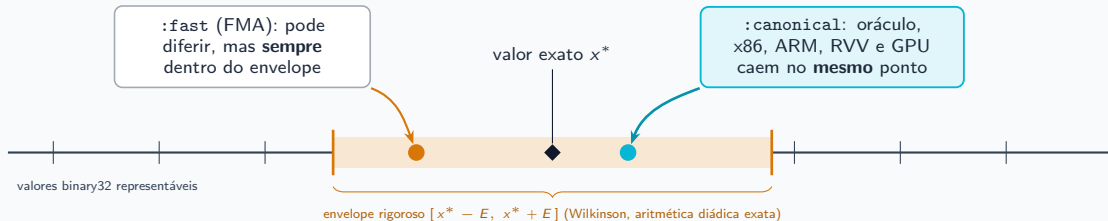
bits × GNU objdump e spirv-val · execução × oráculo exato · x86 × AArch64 × RVV (QEMU, VLEN 128/256/512) × interpretador RVV × lavapipe

Confiado, e nomeado: kernel Linux, BEAM, compilador Zig, kernel do Lean, QEMU, driver Vulkan

Segurança começa por dizer o que *não* está provado. O que não é provado é testado contra um oráculo independente; o que resta é **declarado**, não escondido.

Precisão: exata por construção, envelope quando rápida

2 · POR QUÊ



binary32 na BEAM: floats guardados como **bits**; decodifica $\rightarrow$ binary64 exato

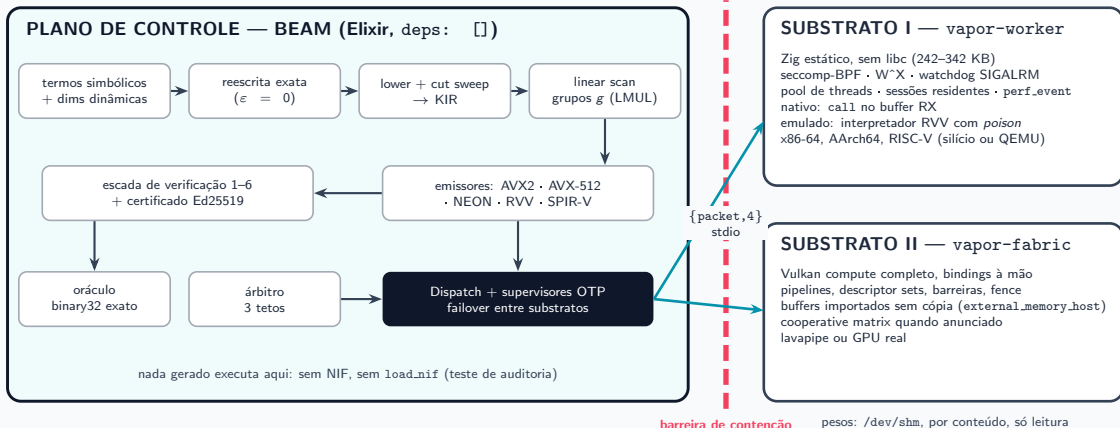
$+$ $-$ $\times$ em **binary64** e arredonda:
 $53 \geq 2 \cdot 24 + 2 \Rightarrow$ arredondamento duplo inócuo

fma: produto diádico exato e um arredondamento, como o hardware

Dois contratos explícitos: `:canonical` = os mesmos bits do oráculo exato em toda máquina; `:fast` = qualquer resultado dentro de um envelope provado, decidido por código extraído do Lean.

Arquitetura tripartite: decidir, executar, sobreviver

3 · COMO

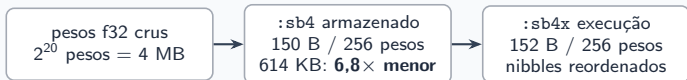


A BEAM **decide, verifica e supervisiona**; os substratos **só executam** — e podem morrer. Nenhum byte gerado roda no espaço de endereçamento da VM.

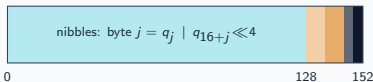
Ponta a ponta: do arquivo de pesos ao registrador e ao token

3 · COMO

DADOS



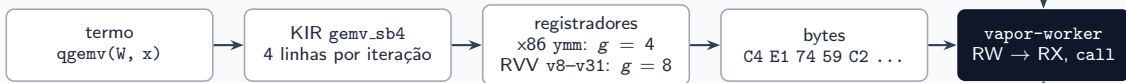
um superbloco sb4x (offsets em bytes)



$u[8], v[8]$ por sub-bloco; c_1, c_0 em f32
 $\tilde{\alpha} = \text{fl}(c_1 u_s), \tilde{\beta} = \text{fl}(c_0 - \tilde{\alpha} v_s)$
 $y = \sum \text{fl}(\tilde{\alpha} q) x + \sum \tilde{\beta} X_s$

pesos via /dev/shm, zero cópia

CÓDIGO



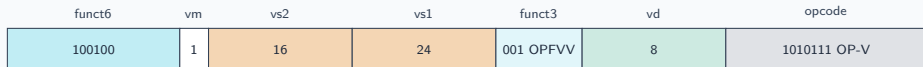
tokens y_t a cada passo: SSM $d = 1024/512$, 64 tokens numa única travessia BEAM↔worker em 10,5 ms, bits idênticos ao oráculo

Um caminho só, visível de ponta a ponta: **sem C, sem LLVM, sem runtime do fornecedor na CPU**. Os pesos cruzam uma vez, por mapeamento; os tokens voltam em fluxo.

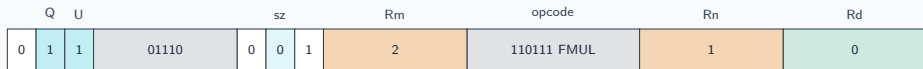
Bits sem toolchain: a mesma multiplicação vetorial em três ISAs

3 · COMO

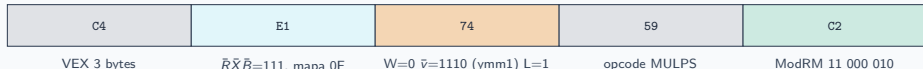
RVV 1.0 `vfmul.vv v8, v16, v24` → `0x930C1457`



AArch64 NEON `fmul v0.4s, v1.4s, v2.4s` → `0x6E22DC20`



x86-64 AVX2 `vmulps ymm0, ymm1, ymm2` → `C4 E1 74 59 C2`



codificador
= função pura
(op, regs) ↦ bits

<<f6::6, vm::1,
vs2::5, vs1::5,
f3::3, vd::5,
0x57::7>>

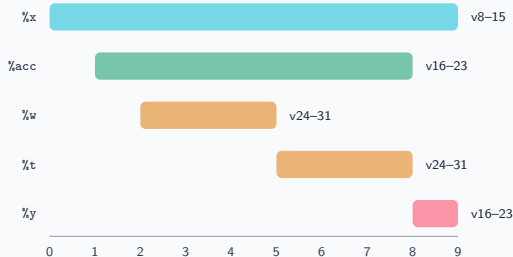
juiz externo
GNU objdump e
spirv-val de-
codificam 100%
das palavras
(todo kernel ×
ISA × política ×
g)

Sem assembler externo, a **especificação da ISA é o único contrato** — e uma ferramenta que não compartilha código conosco é o árbitro. SPIR-V segue igual: palavras de 32 bits, NoContraction para fixar a ordem.

Alocação: linear scan com grupos (LMUL), sem spill, verificada

3 · COMO

intervalos de vida (pontos do programa)



%w e %t não se sobrepõem $\Rightarrow$ reutilizam o grupo; %y nasce quando %acc morre ([s, e)).

banco RVV, $g = 8$: grupos alinhados a múltiplos de 8

v0	v1	v2	v3	v4	v5	v6	v7
v8	v9	v10	v11	v12	v13	v14	v15
v16	v17	v18	v19	v20	v21	v22	v23
v24	v25	v26	v27	v28	v29	v30	v31

v0 = máscara (reservado) $\Rightarrow$ grupo v0-v7 indisponível

só 3 valores de 8 registradores vivos ao mesmo tempo

tenta $g =$
8, 4, 2, 1

depois variante
 $R = 4 \rightarrow 2$
 $\rightarrow 1$ linhas

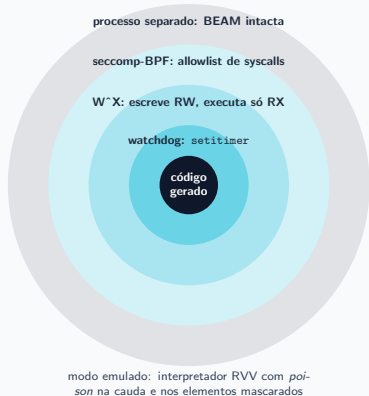
checkAlloc
(Lean) aceita?

aceito: grupos alinhados, dentro do banco, fora dos reservados, **sem sobreposição de vivos** (checkAlloc_sound)

Selecionar antes de alocar, nunca emitir spill, e aceitar só o que um **checker provado** aceita: o alocador é heurístico e pode errar — o código emitido não herda o erro.

Isolamento em camadas: o pior código vira um erro tipado

3 · COMO



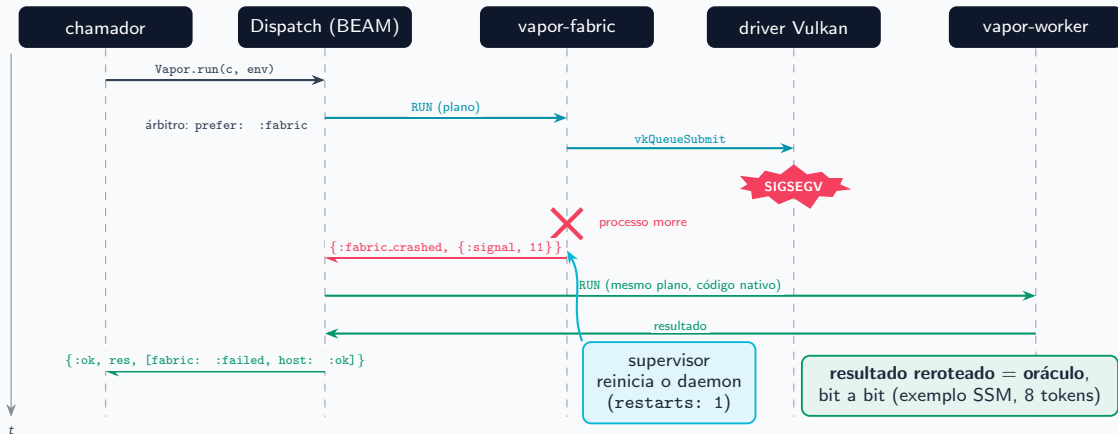
falha injetada	sinal	o que a BEAM recebe
instrução ilegal	SIGILL	<code>{:worker_crashed, {:signal, :sigill}}</code>
acesso inválido	SIGSEGV	<code>{:worker_crashed, {:signal, :sigsegv}}</code>
laço infinito (EB FE)	SIGALRM	<code>{:worker_crashed, {:signal, :sigalrm}}</code>
syscall proibida	SIGSYS	<code>{:worker_crashed, {:signal, :sysys}}</code>
driver Vulkan	SIGSEGV	<code>{:fabric_crashed, {:signal, 11}}</code>

em todos os casos: **a BEAM segue viva**, o processo é reiniciado pelo supervisor e o próximo pedido é atendido — verificado em teste a cada *mix test*

Defesa em profundidade com uma regra simples: **qualquer** coisa que o código gerado faça de errado termina num sinal, e todo sinal termina numa **mensagem**.

Auto-recuperação: a GPU falha no meio do serviço

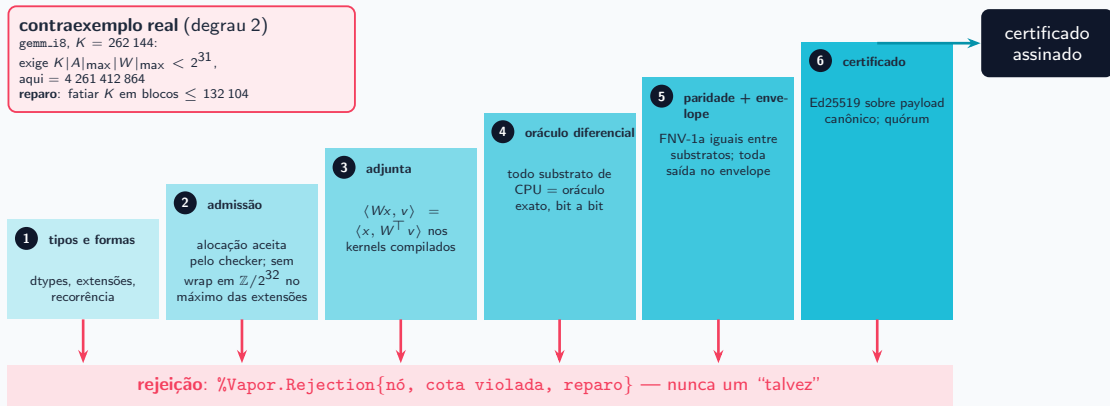
3 · COMO



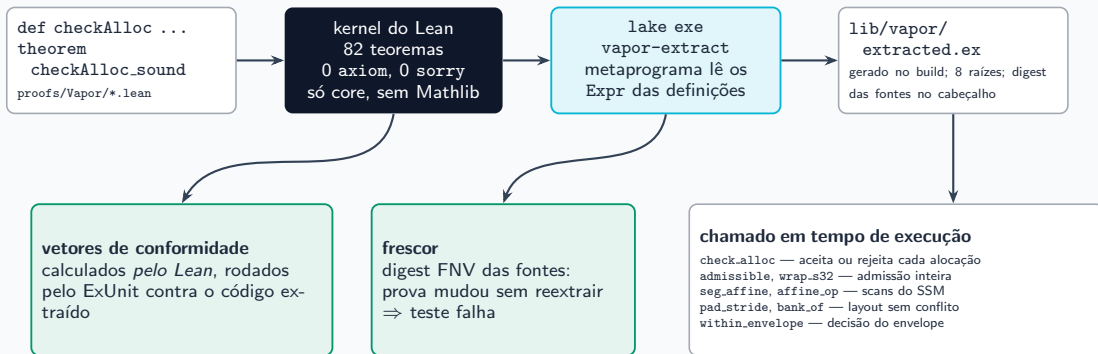
Recuperação é roteamento: como **os bits são os mesmos em todo substrato**, trocar de substrato no meio do serviço não muda a resposta — só o tempo.

A escada de verificação: certificado ou contraexemplo

3 · COMO



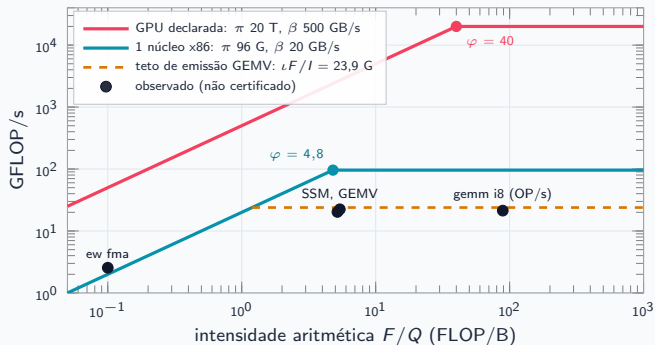
Todo compile termina em **exatamente um** de dois artefatos: um certificado assinado ou um contraexemplo com o reparo. As cotas valem no máximo das extensões e, por monotonia provada, para toda extensão menor.



O código que decide no runtime é **o mesmo termo que o Lean verificou**, traduzido mecanicamente no build. Rodar o Lean dentro da BEAM a cada chamada não acrescentaria garantia — só dependências.

Árbitro: roofline de três tetos, decisão sem profiling

3 · COMO



$$t = \sum \max\left(\frac{F}{\pi}, \frac{Q}{\beta}, \frac{l}{l}\right) + \text{overheads}$$

F, Q do cronograma; l contado no código emitido

decisão = $\arg \min_s t_s$

a crista $\varphi = \pi/\beta$ é reportada mas não decide: com todos os tetos na previsão, exigir $F/Q \geq \varphi$ só escolheria o mais lento

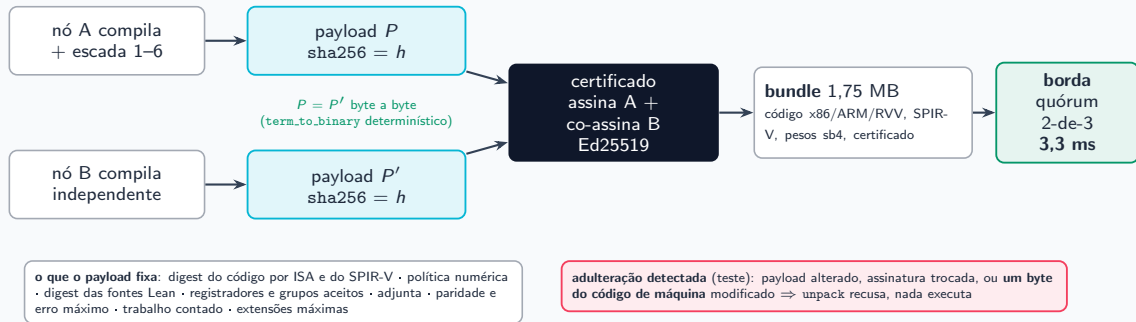
	previsto	observado
GEMV 4096^2 , 32 it.	71,8 ms	73–80 ms
ew fma 1M, 32 it.	33,6 ms	26 ms
SSM, 64 tokens	9,0 ms	10,5 ms
ew fma acima do teto: o β declarado é conservador		

sem o teto l/l o GEMV seria previsto em 17,8 ms: **4 × errado**

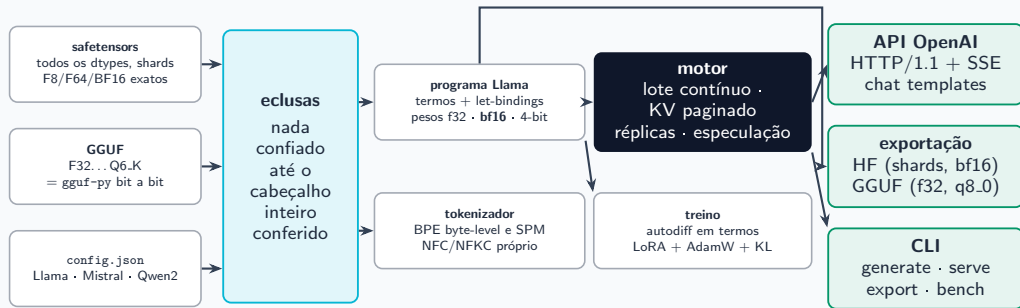
Trabalho **contado**, hardware **declarado**, tempo **previsto**: a mesma entrada dá a mesma decisão em todo nó, sem aquecimento e sem profiling.

Certificado, quórum e borda: verificar custa milissegundos

3 · COMO



Certificar custa segundos (10 s no exemplo) e acontece uma vez; verificar custa **3,3 ms** e acontece em cada borda. A borda herda a confiança sem herdar o custo — nem o toolchain.



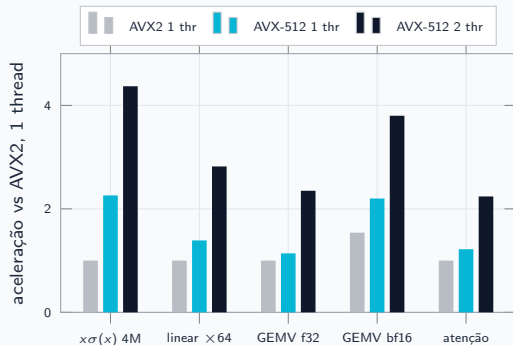
oráculos externos, só nos testes: transformers/torch ($\text{logits} \leq 5,6 \cdot 10^{-7}$; nosso export reproduz o checkpoint bit a bit) · tokenizers (7 configurações × 415 textos idênticos) · gguf-py · llama.cpp (converte → vapor; vapor exporta → llama.cpp) · cliente openai

Nenhuma camada nova pediu exceção às garantias: os modelos são **programas da mesma álgebra** — mesmos kernels, mesma escada, mesmos bits em CPU, RISC-V e GPU.

forma	onde	invariante testado
SIMD	AVX2 · AVX-512 (EVEX, 32 zmm, cauda mascarada) · NEON · RVV 128–512 · SPIR-V	mesmos bits que o oráculo
ILP	GEMV com 4/2/1 linhas por iteração, escolhido pelo alocador	idem
threads	pool no worker; descritor de partição por chamada; decode dividido por cabeça KV	1 = 2 = 3 threads
lote contínuo	decode + pedaços de prefill no mesmo passo	sozinho = em lote, qualquer fatiamento
KV paginado	tabela de blocos, páginas reservadas na admissão	paginado = contíguo
réplicas	uma compilação, mesmas páginas de pesos , menor fila	qualquer réplica; falha isolada
banda	GEMV <i>weight-stationary</i> ; pesos bf16 residentes alargados no kernel	bf16 = f32 sobre pesos arredondados
especulação	rascunho propõe, alvo verifica $k+1$ linhas num passo	saída = alvo sozinho
GPU	SPIR-V de todo operador de modelo	fabric = oráculo (modelo inteiro)
concorrência BEAM	processo por conexão, motor GenServer, SSE; worker morto → :error, sessão reaberta	—

A soma nunca é reassociada: o paralelismo vem de **linhas, sequências, réplicas e cabeças independentes**. Por isso cada forma de HPC entra sem custar a garantia central.

Medido: AVX-512, bf16, threads e o motor



GEMV 2048²; bf16 relativo ao GEMV f32 AVX2. Banda medida: 53 GB/s. docs/bench/BENCH.md, 2026-10-01.

motor (Llama 256, vocab 32k)	seq.	tokens/s
AVX2, 1 thread	8	895
AVX2, 2 threads	8	1 477
2 réplicas $\times$ 1 thread	8	1 555
AVX-512, 2 threads, f32	1	403
AVX-512, 2 threads, bf16	1	588
prefill, 256 tokens	—	~5 500

leitura: kernels de cômputo ganham com AVX-512 ($x\sigma(x)$ 2,3 $\times$); os de banda ficam iguais — o roofline previu; bf16 dobra o GEMV porque lê metade dos bytes; com 1 sequência o motor é limitado por banda (bf16 +46%), com 8 por cômputo. VM de 2 vCPUs, sem PMU: números variam com a carga.

Os ganhos vêm de **onde o roofline diz que estão** — largura vetorial onde há cômputo, bytes a menos onde há banda — e nenhum deles muda um bit da saída.

caminho	juiz	resultado
checkpoint HF → vapor vapor → diretório HF → transformers safetensors de todo dtype	transformers f32 transformers torch, numpy	logits $\leq 5,6 \cdot 10^{-7}$ rel.; greedy idêntico (5 variantes) logits do checkpoint original bit a bit ; bf16 $\leq 10^{-5}$ F8 (os 256 padrões de cada formato), F64, F16, inteiros: bit a bit
f32 → BF16/F16 na escrita	torch	20 000 padrões (empates, subnormais, overflow) bit a bit
HF → <code>convert_hf_to_gguf.py</code> → vapor desquantização GGUF (11 tipos), Q8_0 vapor → GGUF → <code>llama.cpp</code> tokenizador (4 vocabulários reais) servidor	transformers gguf-py libllama tokenizers cliente openai	f32 $5,6 \cdot 10^{-7}$ rel.; q8_0 dentro da quantização bit a bit mesma tokenização; logits f32 $\leq 10^{-4}$, q8_0 $\leq 5\%$ 184 vetores HF + 2 905 textos: idênticos completions e chat, com e sem streaming

Um formato só está suportado quando **a ferramenta de referência concorda** — lendo o que ela escreve e escrevendo o que ela lê. Os juízes rodam só nos testes; o produto continua deps: [].

família	o que exige	como entrou (sem operador aproximado)
Qwen3	normas RMS por cabeça em q/k	linear com matrizes de seleção 0/1
Qwen3-MoE, Mixtral	roteador top- <i>k</i> , especialistas	posto por contagem + sel; soma em ordem fixa de especialista
Gemma 3	GeGLU (tanh), normas (1+ <i>w</i>) e sanduíche, RoPE local/global	novas funções canônicas tanh, gelu_tanh; escala de atenção explícita
DeepSeek-V3	MLA, MoE por grupos (sigmoide), YaRN	MLA como layout de cabeças; grupos por seleção exata; tabelas corretamente arredondadas

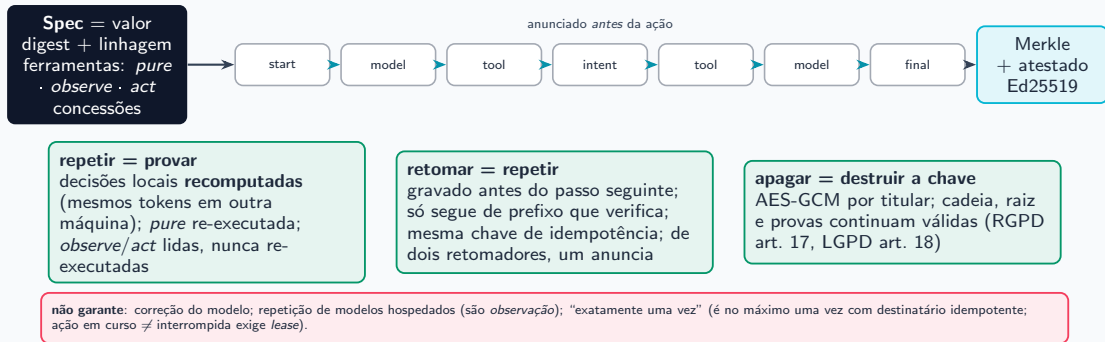
juiz: transformers 5.18 — logits $\leq 8 \cdot 10^{-7}$ rel. em 8 variantes; mesmos bits em AVX2, AVX-512, RVV, QEMU, Vulkan
especialistas *não escolhidos* envenenados com NaN/ ∞ : **nenhum bit muda**; prefill = decode passo a passo

desde 0.6: MoE **esparso** por predicação de linha (bits = denso, $2,6\times$ no decode); MLA com cache **latente** ($85\times$ menos memória); janela exata com cache **circular**. **Recusados pelo nome:** *soft-capping* (Gemma 2), NTK dinâmico, LongRoPE.

Cada peça nova das arquiteturas de fronteira virou **uma contração exata da mesma álgebra** — por isso herdou, sem exceção, os mesmos bits em todo substrato.

Agentes imutáveis: a execução é um objeto de prova

5 · AGENTES, RAG E ZK



O laço de agente é *commodity*; o que só o vapor tem é determinismo — e com ele um agente deixa de produzir **logs** e passa a produzir **provas**.

- ▶ **JSON Schema** e **tool calls** como gramática de bytes sobre a trie do vocabulário: válidos **sem retry**; 0–50 ms/token no vocabulário de 151 936
- ▶ **nenhum beco sem saída**: o teste de propriedade achou 3 (escape no `maxLength`, surrogate solto, UTF-8 malformado) — corrigidos
- ▶ **RAG**: corpus = raiz Merkle; BM25 e densos determinísticos; recibo re-verificável; dentro de `<quote>` só **substrings literais** da fonte
- ▶ **Jinja hermético**: 240 renderizações de 20 templates reais = `jinja2` byte a byte
- ▶ **recibo x-vapor-receipt**: quem tem os pesos re-deriva a resposta

Elixir	veredito
Plug, Phoenix Nx	✓ Vapor.Plug (Bandit, testado) ✓ compilador <code>defn</code> certificado (fragmento)
Livebook Ecto, Oban, LiveView Broadway, EMQX Nerves	✓ passeio executado como teste <i>behaviour</i> Store, <code>on_event</code> , cancelamento ingestão / transporte de eventos viável (workers estáticos); sem placa
AtomVM, Membrane, Riak	✗ com motivo

O núcleo continua deps: `[]`: o ecossistema entra por **pontos de extensão**, e integrar de verdade revolveu defeitos que a descrição escondia (cliente desconectado gerando, *code path* podado).

proposta	escrutínio	feito e testado
RVV para zkVMs (SP1, RISC Zero)	premissa falsa: zkVMs são RV32IM	testemunha pelos kernels certificados, sem zkVM
aritmetizar add/mul/fma	f32 não é aritmética de corpo	int8 $\rightarrow$ R1CS: $16 \rightarrow 8 \rightarrow 3 = 311$ restrições (144 de faixa da entrada); Groth16 ok, forjado rejeitado
verificador on-chain “<200k gás”	depende do verificador	medido: 209 922 gás (execução), 3 entradas públicas
Lean “elimina 100% dos riscos”	exagero: setup, provador, pesos certos	lema provado: $2KAB < p \Rightarrow \text{máquina} = \mathbb{Z} = \mathbb{F}_p$
NTT para FHE e STARK	núcleo comum, vale o kernel	referência exata em 4 corpos, cíclica e negacíclica
Wilkinson = orçamento de ruído ativações por Chebyshev	analogia: ruído CKKS é aleatório Canon não é polinomial puro	— erro provado em racionais (sigmoide gr. 15: $1,388 \cdot 10^{-3}$)
CKKS/BFV na álgebra; zkFHE 70B	cripto sem auditoria; pesquisa	recusado (OpenFHE, Lattigo existem)

Com pesos públicos, o **recibo determinístico** já verifica por reexecução: ZK só se paga por **privacidade** ou **verificação sucinta** — e então o fragmento inteiro, com a ponte provada em Lean, é o caminho barato.

Pilha tradicional × vapor — inclusive onde o vapor perde

	PyTorch / CUDA / vLLM	vapor
Instalação	3,0 GB de wheels (torch 2.14 + CUDA)	■ 1,8 MB no núcleo; 8,7 MB com as 13 rodadas (+ runtime Erlang)
Geração de código	LLVM, NVCC, Triton, ptxas: C++ na base de confiança	■ codificadores puros em Elixir → bits, juiz externo em teste
Falha do driver	no mesmo processo: derruba o serviço	■ processo separado, erro tipado, failover
Determinismo	modo determinístico parcial; muda com GPU e versão	■ :canonical bit-idêntico em x86, ARM, RVV e GPU
Garantia formal	nenhuma sobre o código gerado	■ checker, admissão, envelope provados e extraídos
Artefato de implantação	imagem de container	■ bundle assinado, quórum, verificação em ms
Previsão de desempenho	profiling e autotuning empíricos	■ contada e declarada: mesma decisão em todo nó
RISC-V RVV	fora do caminho principal de otimização	■ alvo de primeira classe, agnóstico a VLEN
Cobertura de operadores	■ milhares de ops, qualquer arquitetura	8 famílias (Llama... DeepSeek-V3), GEMM i8, SSM
Desempenho de pico	■ tensor cores, kernels ajustados à mão	AVX-512, threads; coopmat não executado
Treino e ecossistema	■ autograd completo, distribuído, comunidade	autodiff + LoRA (modelos pequenos)

■ vantagem ■ vantagem do lado oposto

Não é um substituto do PyTorch: é **outro ponto do espaço de projeto** — menos cobertura e menos pico, em troca de garantias que a pilha tradicional não tem como oferecer.

Missão crítica aeroespacial · médica · automotiva

dor

resultado muda com o hardware; base de confiança impossível de auditar

o que ganha hoje

bits idênticos entre plataformas; certificado por compilação; ~19 mil linhas (Elixir + Zig); falha contida

o que ainda falta

certificação regulatória (DO-178C, IEC 62304) não realizada; poucos operadores

Hardware aberto RISC-V RVV 1.0

dor

pilha de ML madura só para x86/CUDA; RVV é segunda classe

o que ganha hoje

código RVV agnóstico a VLEN, validado em 128/256/512; worker estático de 276 KB; interpretador com *poison*

o que ainda falta

validação em silício físico (placas SpacemiT, Milk-V)

Infraestrutura BEAM Elixir · Erlang, alta concorrência

dor

NIF que derruba a VM ou sidecar Python com fila e serialização

o que ganha hoje

deps: `[]`; falha = mensagem; supervisão OTP; *T* passos em 1 travessia com streaming por token

o que ainda falta

escala medida em muitos núcleos; buffers device-local para GPU discreta

Muda o jogo onde **reprodutibilidade**, **auditabilidade** e **tolerância a falhas** valem mais que os últimos 10% de throughput — e diz com clareza o que ainda não entrega.

Rodada 0.6: fronteira, imagem e áudio — sem perder os bits

6 · PARA QUEM

recurso	como (primeiros princípios)	evidência
MoE esperso	predicação por linha no GEMV: não ler pesos que ninguém escolheu	bits = denso; $2,6\times$ no decode
MLA latente	outro programa (absorção), escolhido pelo operador	$85\times$ menos cache; = transformers
janela + anel	anel na <i>tabela de blocos</i> : ordem lógica intacta, $R = \lceil (w+T-1)/p \rceil$	limite justo (controle); $7,9\times$ seqs.
Mamba	a recorrência é a definição; estado realimentado no worker	= transformers; custo/token constante
árvore especulativa	ramos em <i>slots</i> sobre páginas compartilhadas; o vencedor herda as páginas	saída = gulosa; 6 tokens/passo
tensor paralelo	coluna + all-gather (linha + all-reduce muda 81% dos bits)	bits = 1 worker
conv / VAE / DiT	im2col = gather + sel + reshape + GEMV: nenhum kernel novo	$\leq 9 \cdot 10^{-7}$ vs. torch/diffusers
Whisper	encoder + decoder como <i>partes</i> do contrato; K/V cruzados $1\times$	gulosa = transformers
÷ IEEE, Higham	Markstein + Dekker sem FMA (semântica v2); Higham em Lean	0 erros em $5 \cdot 10^7$
log de transparência	RFC 9162 + C2SP, testemunhas, verificação no navegador	196 sondas do transparency-dev

limites: pesos aleatórios — **nenhuma** afirmação de qualidade de imagem, vídeo ou transcrição; sessões residentes na GPU (P0), worker macOS, FlashAttention, ControlNet: não feitos, com motivo (docs/DIRETRIZ.md §9).

Cada truque de serviço que costuma **trocar reprodutibilidade por velocidade** entrou aqui sem essa troca — e com o seu custo medido ao lado.

Rodada 0.7: o escaneado e o LM que sabe se calar

6 · PARA QUEM

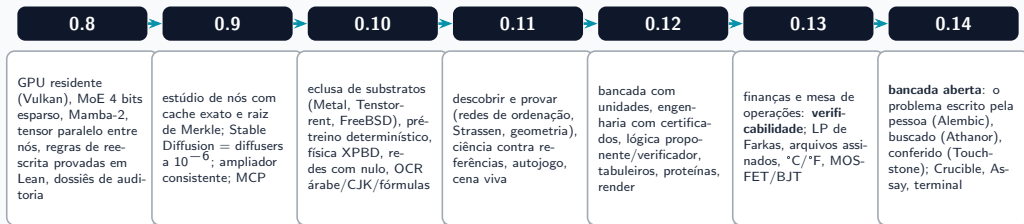
recurso	como (primeiros princípios)	evidência (com controle)
CCITT G3/G4, LZW	T.4/T.6 em Elixir, a máquina de estados do Xpdf	42 fluxos = libtiff bit a bit
ordem de leitura	XY-cut: a <i>calha</i> corta antes do vão; limiares da própria região	8 páginas: CER 1,3% (sem ordem: 53%; Tesseract 1,6%)
feixe + LM	5-gramas Witten–Bell; só escolhe entre o que os quadros acham plausível	retidas 6,5% → 4,4%; embaralhado: nenhum ganho
abstenção	acima de 8 bits/caractere a linha não é língua: o modelo se cala	aleatórias: 0/30 mudadas (sem guarda: 27)
pattern / format	regex ECMA-262 → autômato de bytes UTF-8 com escapes JSON	= re do Python em 7 240 cadeias
fusão em disco	um tensor por vez, mesmos kernels, cabeçalho antes dos dados	mesmos bytes; 4 MB × 83 MB

achado no caminho: máscaras `ImageMask` lidas invertidas; acentos descartados em linhas sem letras altas (“não” lido “rão”); um LM contado sobre Markdown inventava crases em páginas impressas. **limites:** JBIG2, tabelas, manuscrito (`docs/DIRETRIZ.md §10`).

Um modelo de língua é o exemplo clássico de **ruído plausível**: aqui ele só entra com o controle que mostra onde ele *não* deve falar.

De 0.8 a 0.14: um domínio por vez — e então a bancada aberta

7 · RODADAS



a regra que atravessa as rodadas: uma busca *propõe*, um verificador *decide*; cada resposta traz o número que poderia ter saído de outro jeito (o controle). §5 de **QUALITY.md**: 0.11 → 122, 0.12 → 147, 0.13 → +23, 0.14 → +20 verificações com controle.

A amplitude cresceu sem que o núcleo mudasse de natureza: **cada domínio novo entra pela mesma porta — a do certificado.**

0.13: finanças — verificabilidade, não velocidade

7 · RODADAS

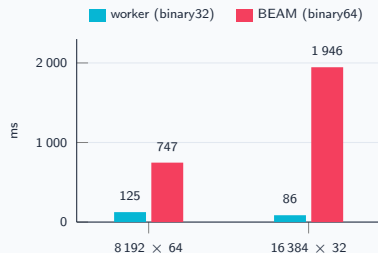
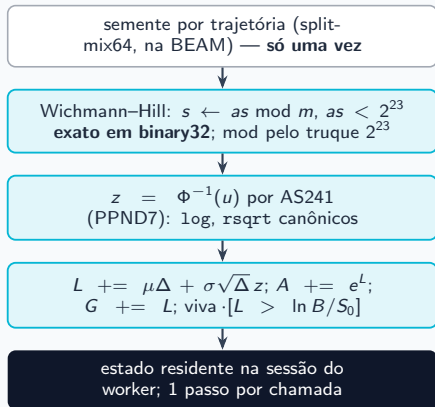
tarefa	o que entra (texto da mesa)	o certificado que sai
calendários, dinheiro	du 2025-01-02 2026-01-02, allocate 100.00 1 1 1	feriados por regra (cômputo) = QuantLib día a día, 1990–2078 ; rateio que soma exatamente
curvas	DI1, LTN, NTN-F, depósitos, swaps	todo instrumento reprecificado ($4,5 \cdot 10^{-16}$); forwards negativos apontados
opções	price/iv/american/heston/smile	paridade; gregas $\times$ dif. finitas; limites de não arbitragem <i>antes</i> , $g(k) \geq 0$ de Durrleman; = QuantLib (10^{-9} – 10^{-12})
Monte Carlo	S, K, T, r , σ , trajetórias, barreira	bits = oráculo = 2 threads; IC de 99% $\times$ forma fechada
risco	P&L e método de VaR	Kupiec, Christoffersen, Basileia; tamanho e poder do teste medidos
backtest	dados, varredura, sinal causal, custo	quatro portões : prefixo, DSR, PBO, Reality Check
arbitragem	estados / calls / câmbio	um portfólio ou preços de estado , em $\mathbb{Q}$
livro de ofertas	buy 100 @ 101.25 gtc owner=a	diário SHA-256 + Merkle; juiz ingênuo ; ITCH; FIX

escrutínio: “HFT” lido literalmente é um motor de nanossegundos (FPGA, *kernel bypass*); a BEAM é tempo real brando. Entregue: o que de HFT *se confere* — e a latência medida ($\sim 8,5 \mu\text{s}$ /evento com o hash).

Um preço que muda de máquina para máquina, um backtest que olhou o amanhã, a melhor de 50 estratégias vendida como a única: **falhas de prova, não de desempenho**.

Monte Carlo canônico: o gerador dentro da álgebra

7 · RODADAS



oráculo (64 pistas)

2 threads

BSM em IC 99%

controle: sem ltô

asiática c/ var. de controle

✓ bits iguais

✓ bits iguais

✓ $z = 1,17$

✗ $z = 8,7$ — pego

variância $\div 1300$

Um número de risco que **não muda com a máquina nem com as threads** — e que diz isso na própria resposta.

Backtests: antecipação é propriedade de prefixo

7 · RODADAS



o sinal sobre cada prefixo tem de ser **igual, bit a bit**, ao sinal da história inteira até o corte. Caixa-preta: não confia nos operadores — e pega o center (causal a cada passo, não causal no todo).

	look-ahead	DSR	PBO	RC p
ruído: melhor de 30 cruzamentos	✓	0,10	0,24	0,78
plantado: AR(1), $\phi = 0,15$	✓	1,00	< 0,5	< 0,05
sign(lead(close)-close)	dia 89	—	—	—
sign(center(close))	pego	—	—	—

DSR (Bailey & López de Prado): $P[\text{SR} > \text{SR}_0]$, com
 $\text{SR}_0 = \sqrt{V}((1-\gamma)\Phi^{-1}(1-\frac{1}{N}) + \gamma\Phi^{-1}(1-\frac{1}{N_e}))$
PBO por CSCV: 12870 metades de 16 blocos
RC de White com bootstrap estacionário

O Sharpe do trapaceiro é 20; o do ruído selecionado parece bom. Os portões dizem os dois.

cotações bid/ask e *payoffs* por estado

ou um portfólio
custo < 0 , *payoff* ≥ 0

ou preços de estado
 $\psi > 0$, bid
 $\leq X\psi \leq \text{ask}$

simplex em $\mathbb{Q}$ (duas fases, Bland). Nunca os dois, nunca nenhum. O certificado é **conferido só multiplicando**.

cotações

veredito exato

binomial coerente
call a 14–14,5

$\psi = (1/2, 9/20)$
vender 1 título, comprar 1/90 da ação,
vender 1/60 da call: **recebe 1/15**, paga 0
nos dois estados

calls 90/100/110

borboleta $\frac{1}{2} C_{90} - C_{100} + \frac{1}{2} C_{110}$: **recebe 1/4**

câmbio USD/EUR/BRL

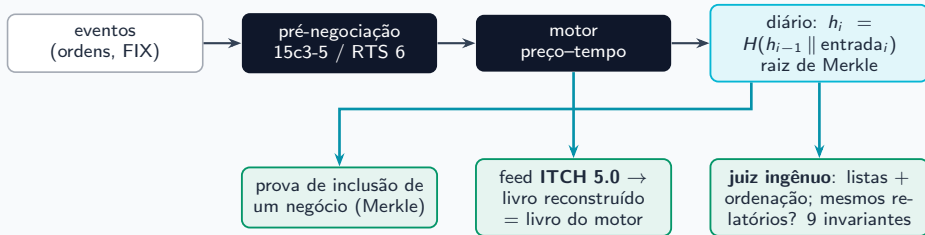
ciclo com produto 590000/584821 > 1

humano ou IA no loop: `arbitrage.check` (MCP) recebe a *proposta* de qualquer um; “compre a ação e venda a call” paga nos dois estados mas custa 86,5 — **rejeitada, com o motivo**.

Arbitragem não é estimada: é **decidida**, e a decisão vem com o objeto que a prova.

O livro de ofertas como objeto verificável

7 · RODADAS



o que o juiz achou: uma ordem FOK a mercado virava IOC — nos dois motores, porque a mesma leitura errada da especificação foi escrita duas vezes. Só o invariante "FOK é tudo-ou-nada", que não executa nada, viu. Fuzzing: 6000 eventos por política; um negócio forjado com o hash recalculado passa na cadeia e cai na reexecução.

Dois motores não bastam: os invariantes são o terceiro juiz. E o backtest roda o código da bolsa, não um modelo dela.

Hawkes

plantado $(\mu, \alpha, \beta) = (1; 0,6; 1,5)$
ajustado $(1,08; 0,61; 1,69)$
ramificação $0,36 (0,40)$

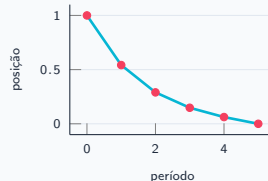
teste de reescala	KS p
Hawkes	0,73
Poisson (controle)	$1,7 \cdot 10^{-20}$

Avellaneda–Stoikov

$\gamma = 0,1$, 1 000 trajetórias, §4 do artigo

	daqui	artigo
$\sigma(\text{P\&L})$ estoque	6,44	5,89
$\sigma(\text{P\&L})$ simétrica	13,57	13,43
$\sigma(q)$ estoque	2,97	2,80
$\sigma(q)$ simétrica	8,37	8,66

Almgren–Chriss



fechada (linha) = numérica (pontos): 10^{-16}

Um modelo de microestrutura sem o seu teste é uma história; **com o teste e o controle, é uma medida.**

pendência	fechada como
LP com Farkas	simplex em $\mathbb{Q}$: dual, Farkas, raio; = HiGHS a 10^{-9}
arquivos assinados	Ed25519 sobre o manifesto; a mentira coerente recusada
$^{\circ}\text{C}$ / $^{\circ}\text{F}$	leituras afins; $98,6\text{ }^{\circ}\text{F} = 37\text{ }^{\circ}\text{C}$; compostas recusadas
transistores	MOSFET nível 1 e Ebers–Moll = ngspice na 6ª casa

aberto

motor de ofertas no worker (latência de máquina)
dados reais (ITCH da Nasdaq, curvas ANBIMA): precisam de rede
gerador inteiro (Philox) no compilador: falta mulhi
Hull–White, LMM, Dupire, XVA, crédito
juiz ingênuo em outra linguagem

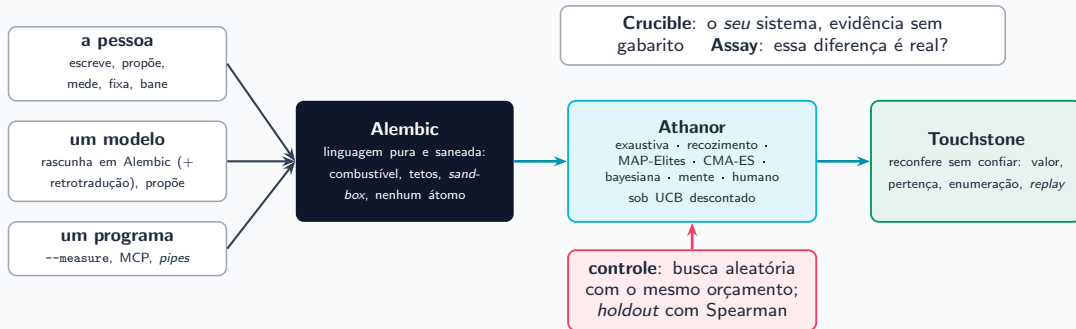
medido nesta rodada

§5i: 23/23 verificações com controle (25 s)
console: 40 exemplos $\times$ 2 línguas no Chromium (87 conferências)
oráculos: QuantLib, SciPy, simplefix, ngspice

As pendências fechadas são as que o domínio novo **tornou necessárias**: o simplex decide a arbitragem; a assinatura faz de uma sessão um registro.

0.14: de vitrine a ferramenta

7 · RODADAS



os buscadores famosos são casos de *proponha* → *avalie* → *certifique*: o produto é o caso geral, e os casos viram **exemplos** (Golomb, Ramsey, redes de ordenação, portfólios, hiperparâmetros, regras de *trading*, fórmulas)

Ninguém escolhe de uma lista: **escreve o problema**. O modelo propõe; **quem decide é o verificador** — o mesmo para a pessoa, o modelo e o acaso.

problema	fornalha	controle
Golomb, 7 marcas	25 (ótimo)	aleatório: 0 acertos em 3923
max-cut, 2^{10} cortes	13, exaurido	força bruta independente: 13; forjado 14: recusado
$R(3, 3) = 6$	provado em 32768 grafos	K_5 : o pentágono refuta
Euler (somas de potências)	contraexemplo em 40	—
médias móveis, <i>holdout</i>	$AR(1)$ $\varphi = 0,5$: $\rho = 0,73$	passeio aleatório: $\rho = -0,33$
replay	mesma semente, mesma raiz	semente 2: outra raiz



a fornalha ao vivo: faíscas por avaliação, a linha do melhor, a tracejada do controle, a partilha do portfólio

“Provado” quer dizer **enumeração completa de um espaço finito**, com o tamanho no certificado; o resto é evidência, e o *holdout* diz quando é viés de seleção.

Crucible e Assay: evidência sem gabarito; sinal ou ruído

7 · RODADAS

Crucible (o sistema é da pessoa)

			valor	controle
6 hamiltonianos aleatórios	cúbicos	H	6/6 provado sobre $\mathbb{Q}$	dissipativos: 0 leis
oscilador quântico		$ E - (n + \frac{1}{2}) $	$3 \cdot 10^{-6}$, ordem 1,99	caixa pequena: evidência falha
Kepler, 250 órbitas		Yoshida: $\leq 1,4 \cdot 10^{-3}$		RK4: 0,062
H ₂ RHF/STO-3G			-1,1167	esticado: acima de 2 H (falha do RHF, exibida)
regressão simbólica		$R^2 = 1,0$		alvo embaralhado: -0,32

Assay (pesquisa em IA)

			valor	controle
comparação pareada	erro tipo I	0,00 em 60 nulos		efeito plantado: $p = 2,5 \cdot 10^{-4}$
calibração	calibrado: $p = 0,45$			confiante: $p = 0,002$; Platt 0,23 → 0,08
Krippendorff		0,743 (publicado)		ao acaso: 0,007
lei de escala	$\alpha \in [0,334; 0,351]$, erro 0,3 %			perdas embaralhadas: 33 %

o controle achou um defeito: com perdas embaralhadas o ajuste es-tourava exp. Agora é finito e o *holdout* diz que a lei não prevê nada.

A pergunta mais comum da pesquisa em IA não é “como treinar”: é “**essa diferença é real?**”.

```
$ vapor athanor run euler.alb > c.json # sai 1: refutada
$ vapor verify euler.alb c.json --replay # sai 0: conferido
$ echo "régua de Golomb com 8 marcas" \
  | vapor mind formalize - > g.alb # + retrotradução
$ vapor athanor run hp.alb --measure ./treina.sh
$ vapor crucible laws sir.txt | jq '.laws[].law'
$ vapor assay compare evals.csv && echo real
$ vapor scene edit s.json "add glow sol { x: 0.7 }"
```

JSON em *pipes*; saídas 0 positivo · 1 negativo · 2 uso · 3 entrada · 4 falha

os mesmos verbos

CLI (bin/vapor) · console (Espaço de trabalho, Crisol, Ensaio) · TUI · MCP (20 ferramentas: athanor_run aceita proposals de um agente)

identidade

fuligem, pergaminho, latão, verdete, cinábrio; a **fornalha** é o gráfico da busca e a **pedra de toque** é o veredito (ouro: passou; chumbo: falhou)

medido

§5j: 20/20 verificações com controle (60 s); 87 testes da rodada; os exemplos do console no Chromium

Nenhuma capacidade existe só na interface gráfica: **o console é um cliente do terminal**, não o contrário.

Legado: o que fica, com a evidência de cada item

8 · CONTRIBUIÇÃO

1

Tensor → bits em 5 alvos sem toolchain

codificadores puros; objdump e spirv-val aceitam 100% das palavras

2

Contenção total

nenhum byte gerado na VM; 5 classes de falha viram mensagens (testadas)

3

Numérica canônica portátil

mesmos bits em x86, ARM, RVV (qualquer VLEN) e Vulkan

4

Provas que executam

82 teoremas, 0 axiomas; Lean → Elixir extraído, com frescor por digest

5

Desempenho previsível

três tetos, trabalho contado do código: erro $\leq 30\%$ sem nenhuma medição

6

LLM, imagem, áudio e agentes no mesmo núcleo

fronteira, SSM, VAE/DiT, Whisper conferidos; agentes cuja execução é prova

7

Domínios com certificado

engenharia, lógica, ciência, finanças: cada resposta com o seu controle

limites, sem maquiagem

sem silício RVV nem GPU discreta nos testes (QEMU, lavapipe)

Metal e FreeBSD compilados, não executados

VM de 2 vCPUs sem PMU

mesa de operações na BEAM: μs , não ns

próximos passos

medir as sessões residentes numa GPU real

rodar a suíte em placa RVV 1.0 e num Mac

multiplicação inteira na álgebra (NTT, Philox)

motor de ofertas como programa do worker

A contribuição permanente não é velocidade: é mostrar que um compilador de tensores pode ser **pequeno, isolado, portátil bit a bit** — e carregar a própria prova.